

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

COURSE NAME: Fundamentals of Data Science

COURSE CODE: DSA0401

COURSE OUTCOME COVERED

***CO1:** Analyse the role of data science, facets of data, and the big data ecosystem for informed decision-making. (BL2)*

QUESTIONS: 05

Total Marks:100

Annexure A

Exploratory Data Analysis - Based Practical Assessment

Code of Conduct:

*I Shaik Mohammed Abrar (192472185) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Shaik Mohammed Abrar

RUBRICS FOR CALCULATION

Exploratory Data Analysis-Based Practical Assessment					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Dataset Understanding	20	Clearly explains dataset source, features, target variables, data types, and problem context with strong clarity	Explains dataset, features, and variables with adequate clarity	Basic explanation of dataset with limited understanding of variables	Poor understanding of dataset; variables and problem context are unclear
Data Cleaning	25	Effectively identifies and handles missing values, duplicates, incorrect entries, and outliers using suitable methods	Handles most missing values, duplicates, and errors correctly	Performs basic cleaning but misses some important data quality issues	Minimal or incorrect data cleaning; major errors remain in the dataset
Data Analysis and Statistical Summary	20	Provides detailed descriptive statistics, comparisons, patterns, and meaningful interpretations	Provides relevant summary statistics and basic interpretation	Provides limited statistics with weak explanation	Lacks statistical analysis or gives incorrect interpretation
Data Visualization	20	Uses highly appropriate charts with proper labels, titles, and clear visual interpretation	Uses suitable charts with mostly clear labels and explanation	Uses limited or basic charts with unclear interpretation	Poor or incorrect visualizations; charts do not support analysis
Insight Generation and Reporting	15	Presents strong insights, conclusions, and recommendations based on data evidence	Presents clear findings and conclusions with reasonable support	Provides basic findings but lacks depth or proper justification	Findings are unclear, unsupported, or not related to the data

CO1-ASSESSMENT TOOL -2 Concept Map**QUESTIONS : 5****MARKS : 50**

Q No	Dataset Understanding (20%)	Data Cleaning (25%)	Data Analysis and Statistical Summary (20%)	Data Visualization (20%)	Insight Generation and Reporting (15%)	Total Score (100%)
1	/2.5	/2.5	/2.0	/2.0	/1.0	/10
2	/2.5	/2.5	/2.0	/2.0	/1.0	/10
3	/2.5	/2.5	/2.0	/2.0	/1.0	/10
4	/2.5	/2.5	/2.0	/2.0	/1.0	/10
5	/2.5	/2.5	/2.0	/2.0	/1.0	/10
Overall Rubrics Value	/12.5	/12.5	/10	/10	/10	/50
	/25	/25	/20	/20	/10	/100

1. Exploratory Data Analysis (EDA) of Student Performance Dataset

```
# =====
# Simple Exploratory Data Analysis (EDA)
# =====

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load Dataset
df = pd.read_csv("Student_Performance_Dataset.csv")

# Display Dataset
print(df.head())
print(df.info())
print("\nShape:", df.shape)
print("\nMissing Values:\n", df.isnull().sum())
print("\nDuplicate Records:", df.duplicated().sum())

# Statistical Summary
print("\nStatistical Summary")
print(df.describe())

# Result Distribution
print("\nResult Count")
print(df["Result"].value_counts())

# Average Values by Result
print("\nAverage by Result")
print(df.groupby("Result")[["Attendance",
                             "Study_Hours",
                             "Internal_Marks"]].mean())

# Correlation
df["Final_Result"] = df["Result"].map({"Fail":0,"Pass":1})

print("\nCorrelation Matrix")
print(df[["Attendance",
          "Study_Hours",
          "Internal_Marks",
          "Final_Result"]].corr())

# ----- Visualization -----

# Histogram
df[["Attendance", "Study_Hours", "Internal_Marks"]].hist(figsize=(8,5))
plt.show()

# Scatter Plot
```

```

sns.scatterplot(data=df,
                x="Attendance",
                y="Internal_Marks",
                hue="Result")
plt.show()

# Box Plot
plt.figure(figsize=(8,4))
for i, col in enumerate(["Attendance",
                        "Study_Hours",
                        "Internal_Marks"]):
    plt.subplot(1,3,i+1)
    sns.boxplot(y=df[col])
    plt.title(col)

plt.tight_layout()
plt.show()

# Heatmap
plt.figure(figsize=(5,4))
sns.heatmap(df[["Attendance",
                "Study_Hours",
                "Internal_Marks",
                "Final_Result"]].corr(),
            annot=True,
            cmap="coolwarm")
plt.show()

# Top Students
print("\nTop 5 Students")
print(df.nlargest(5, "Internal_Marks")[["Student_ID",
                                        "Attendance",
                                        "Study_Hours",
                                        "Internal_Marks"]])

print("\nEDA Completed Successfully.")

```

OUTPUT

```

  Student_ID  Attendance  Study_Hours  Internal_Marks  Result
0         S01           92            5              45    Pass
1         S02           55            2              24    Fail
2         S03           78            4              38    Pass
3         S04           60            3              28    Fail
4         S05           88            5              42    Pass
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  -
0   Student_ID      20 non-null    object
1   Attendance      20 non-null    int64
2   Study_Hours     20 non-null    int64
3   Internal_Marks  20 non-null    int64

```

```
4    Result          20 non-null    object
dtypes: int64(3), object(2)
memory usage: 932.0+ bytes
None
```

```
Shape: (20, 5)
```

```
Missing Values:
```

```
Student_ID      0
Attendance      0
Study_Hours     0
Internal_Marks  0
Result          0
dtype: int64
```

```
Duplicate Records: 0
```

```
Statistical Summary
```

	Attendance	Study_Hours	Internal_Marks
count	20.000000	20.000000	20.000000
mean	70.150000	3.350000	32.950000
std	15.938286	1.424411	9.087151
min	45.000000	1.000000	18.000000
25%	57.250000	2.000000	24.750000
50%	70.000000	3.000000	33.500000
75%	83.500000	4.250000	40.250000
max	95.000000	6.000000	48.000000

```
Result Count
```

```
Result
```

```
Pass      11
```

```
Fail       9
```

```
Name: count, dtype: int64
```

```
Average by Result
```

	Attendance	Study_Hours	Internal_Marks
Result			
Fail	55.111111	2.111111	24.333333
Pass	82.454545	4.363636	40.000000

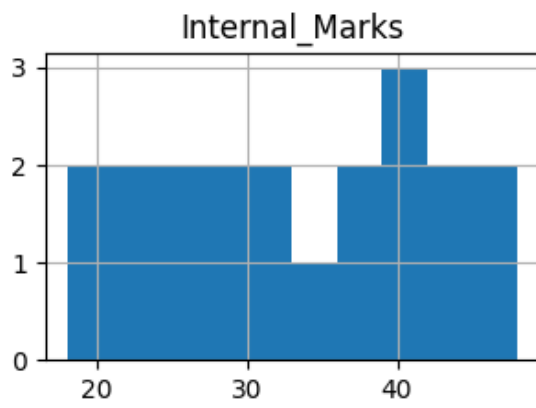
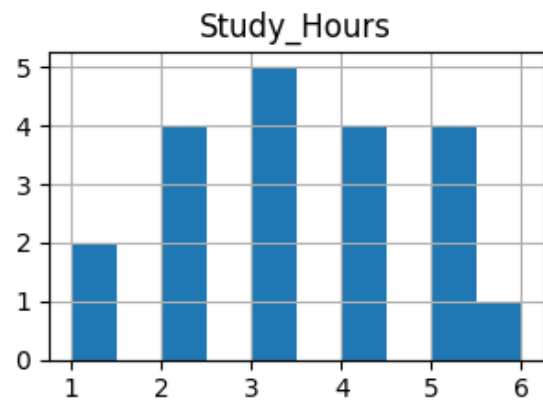
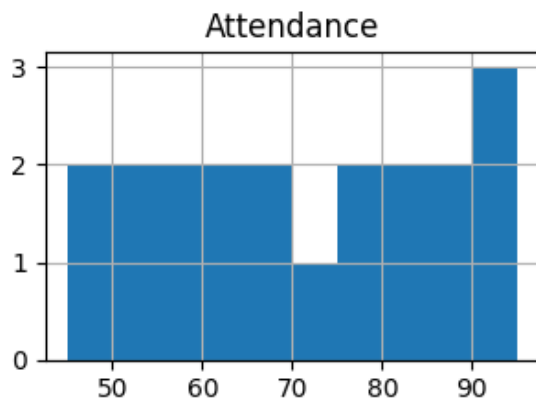
```
Correlation Matrix
```

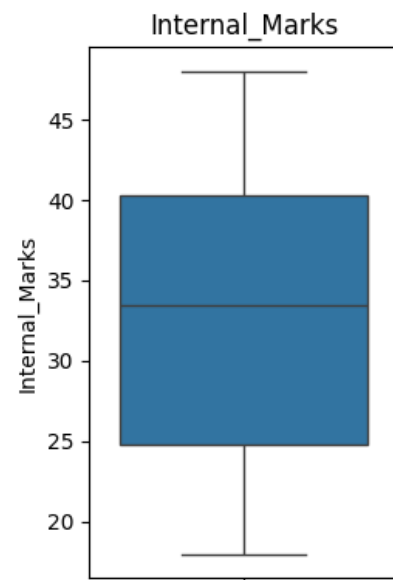
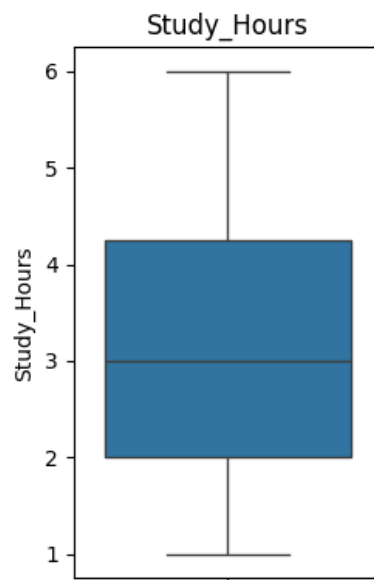
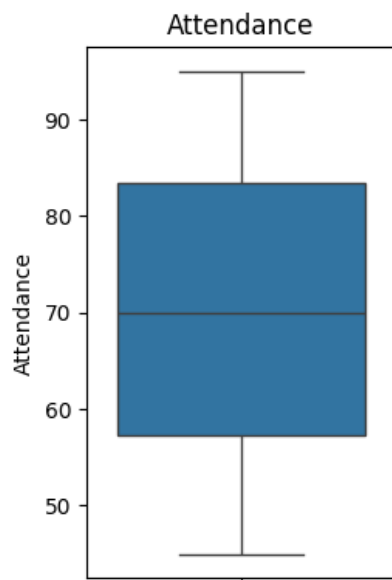
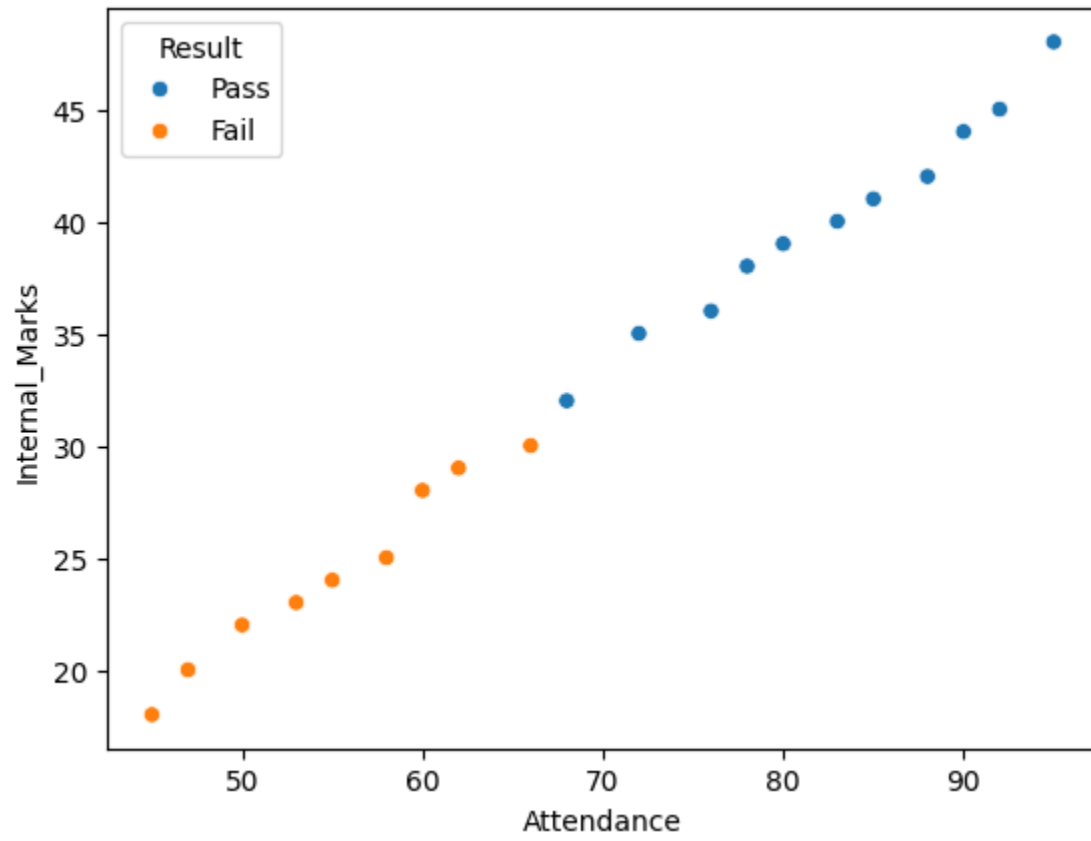
	Attendance	Study_Hours	Internal_Marks	Final_Result
Attendance	1.000000	0.975888	0.997569	0.875664
Study_Hours	0.975888	1.000000	0.977299	0.807161
Internal_Marks	0.997569	0.977299	1.000000	0.879984
Final_Result	0.875664	0.807161	0.879984	1.000000

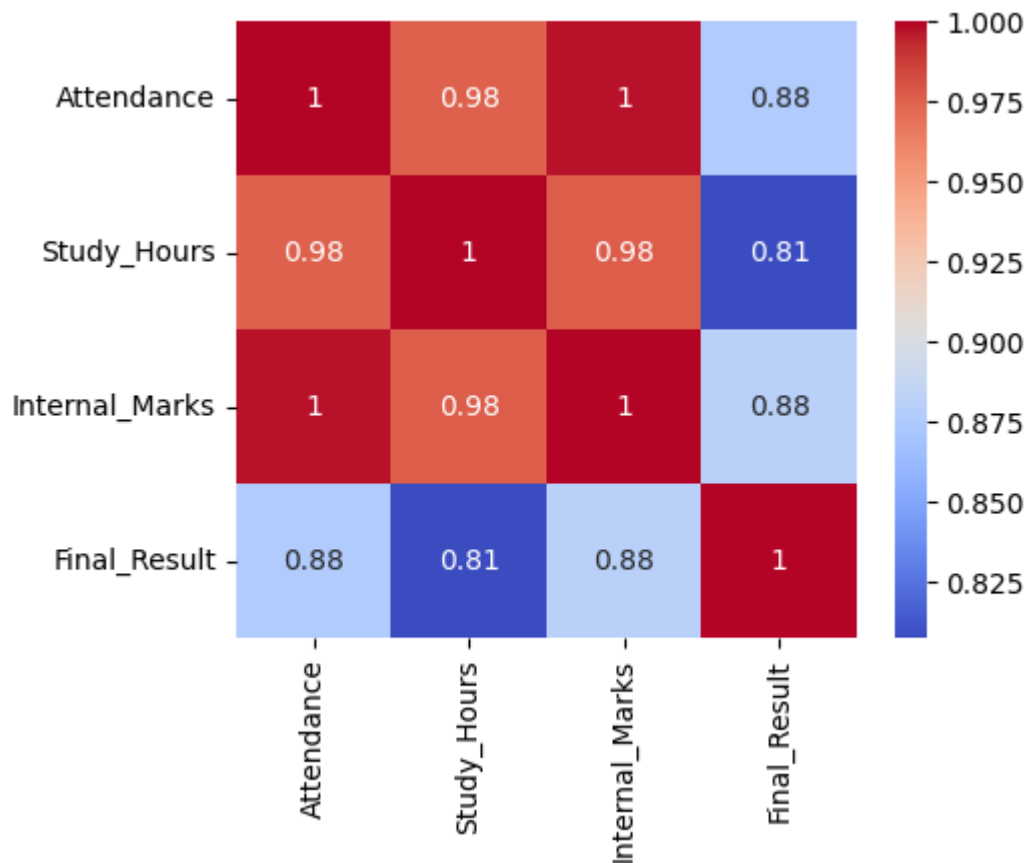
```
Top 5 Students
```

	Student_ID	Attendance	Study_Hours	Internal_Marks
6	S07	95	6	48
0	S01	92	5	45
11	S12	90	5	44
4	S05	88	5	42
15	S16	85	5	41

```
EDA Completed Successfully.
```







2. Hospital Patient Health

```
# =====
# Simple EDA - Hospital Patient Data
# =====

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load Dataset
df = pd.read_csv("Patient_Dataset.csv")

# Dataset Overview
print(df.head())
print(df.info())
print("\nShape:", df.shape)
print("\nMissing Values:\n", df.isnull().sum())
print("\nDuplicate Records:", df.duplicated().sum())

# Statistical Summary
print("\nStatistical Summary")
print(df.describe())

# Disease Distribution
print("\nDisease Count")
print(df["Disease_Status"].value_counts())
```

```

# Average Values by Disease Status
print("\nAverage Values")
print(df.groupby("Disease_Status")[["Age", "Sugar_Level", "BP", "BMI"]].mean())

# Correlation
df["Disease"] = df["Disease_Status"].map({"No":0, "Yes":1})

print("\nCorrelation Matrix")
print(df[["Age", "Sugar_Level", "BP", "BMI", "Disease"]].corr())

# ----- Visualizations -----

# Histograms
df[["Age", "Sugar_Level", "BP", "BMI"]].hist(figsize=(8,6))
plt.show()

# Disease Count
sns.countplot(x="Disease_Status", data=df)
plt.title("Disease Status")
plt.show()

# Boxplot
plt.figure(figsize=(8,4))
for i, col in enumerate(["Age", "Sugar_Level", "BP", "BMI"]):
    plt.subplot(1,4,i+1)
    sns.boxplot(y=df[col])
    plt.title(col)

plt.tight_layout()
plt.show()

# Scatter Plot
sns.scatterplot(data=df,
                x="Sugar_Level",
                y="BMI",
                hue="Disease_Status")
plt.title("Sugar Level vs BMI")
plt.show()

# Heatmap
plt.figure(figsize=(5,4))
sns.heatmap(df[["Age", "Sugar_Level", "BP", "BMI", "Disease"]].corr(),
            annot=True,
            cmap="coolwarm")
plt.show()

# Top Patients with Highest Sugar Level
print("\nTop 5 Patients")
print(df.nlargest(5, "Sugar_Level")[["Patient_ID",

```

```

    "Age",
    "Sugar_Level",
    "BP",
    "BMI"]])

```

```

print("\nEDA Completed Successfully.")

```

OUTPUT

```

    Patient_ID  Age  Sugar_Level  BP  BMI  Disease_Status
0         P01   45         145  130   28             Yes
1         P02   32          98  118   23             No
2         P03   55         180  145   31             Yes
3         P04   28          90  110   22             No
4         P05   60         200  150   34             Yes

```

```

<class 'pandas.core.frame.DataFrame'>

```

```

RangeIndex: 20 entries, 0 to 19

```

```

Data columns (total 6 columns):

```

```

#      Column      Non-Null Count  Dtype
---  -
0      Patient_ID    20 non-null    object
1      Age           20 non-null    int64
2      Sugar_Level    20 non-null    int64
3      BP            20 non-null    int64
4      BMI           20 non-null    int64
5      Disease_Status 20 non-null    object

```

```

dtypes: int64(4), object(2)

```

```

memory usage: 1.1+ KB

```

```

None

```

```

Shape: (20, 6)

```

```

Missing Values:

```

```

    Patient_ID    0
Age              0
Sugar_Level      0
BP               0
BMI              0
Disease_Status   0
dtype: int64

```

```

Duplicate Records: 0

```

```

Statistical Summary

```

```

      Age  Sugar_Level  BP  BMI
count  20.000000    20.00000  20.000000  20.000000
mean   43.200000   138.65000  129.750000  27.300000
std    11.316499    40.33449   14.523574   4.256265
min    27.000000    88.00000  108.000000  21.000000
25%    33.500000    99.50000  117.500000  23.750000
50%    43.500000   137.50000  129.000000  27.500000
75%    52.250000   171.25000  142.500000  30.250000
max    62.000000   210.00000  155.000000  35.000000

```

```

Disease Count

```

```

Disease_Status

```

```

Yes      10

```

```

No       10

```

```

Name: count, dtype: int64

```

Average Values

	Age	Sugar_Level	BP	BMI
Disease_Status				
No	33.5	103.3	117.4	23.7
Yes	52.9	174.0	142.1	30.9

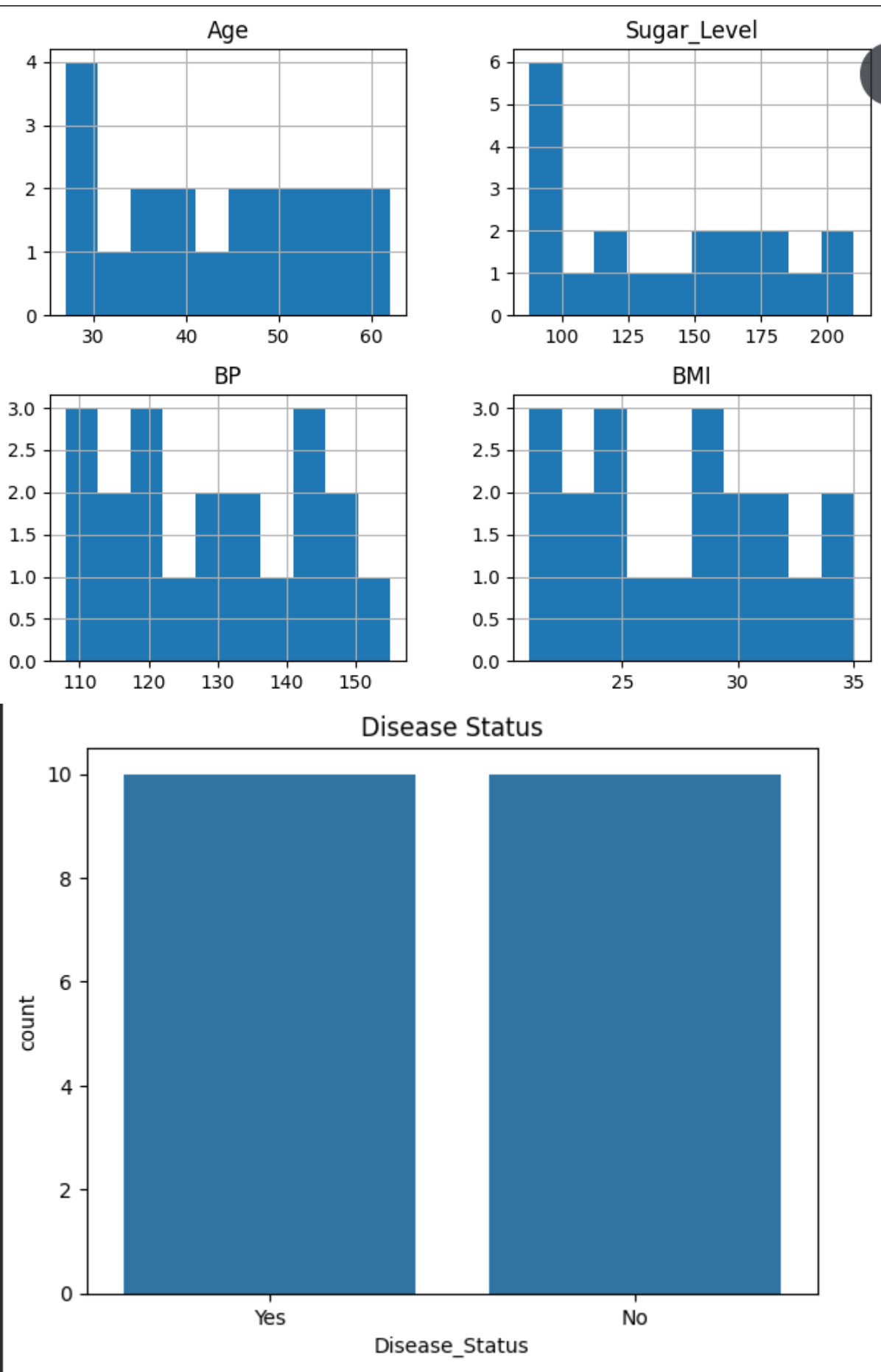
Correlation Matrix

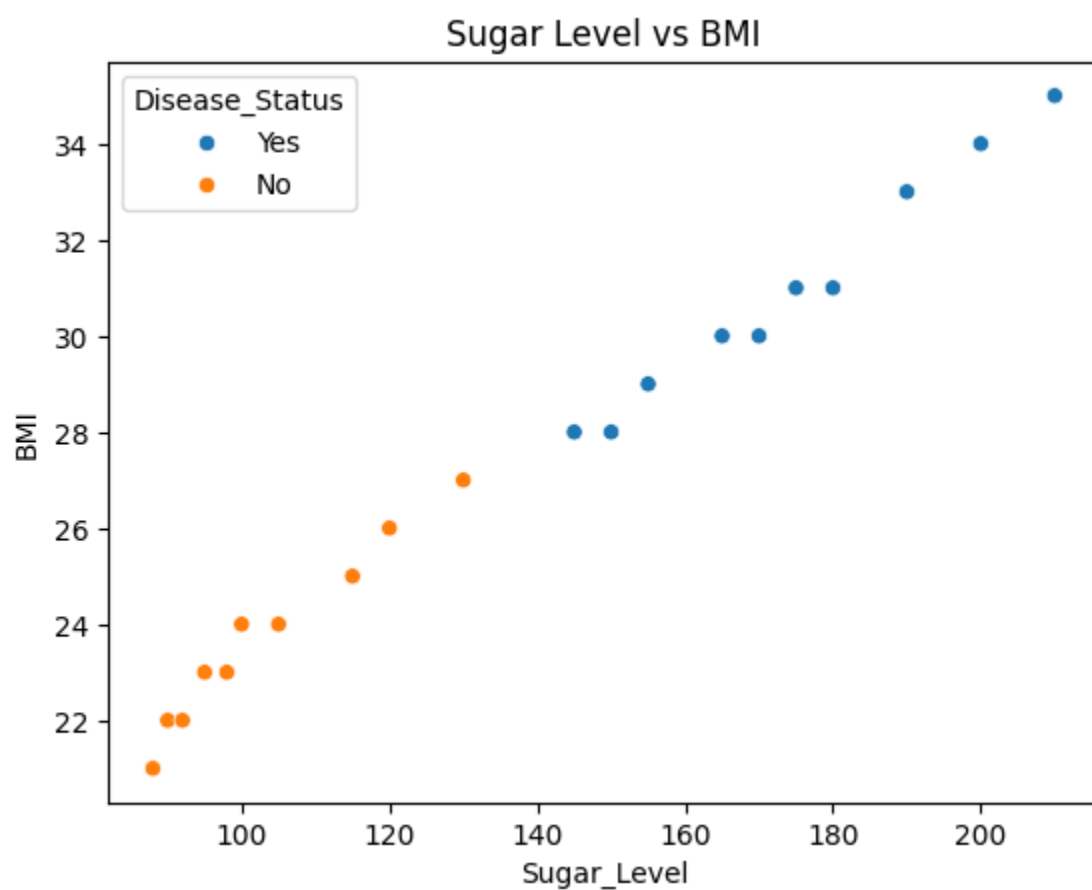
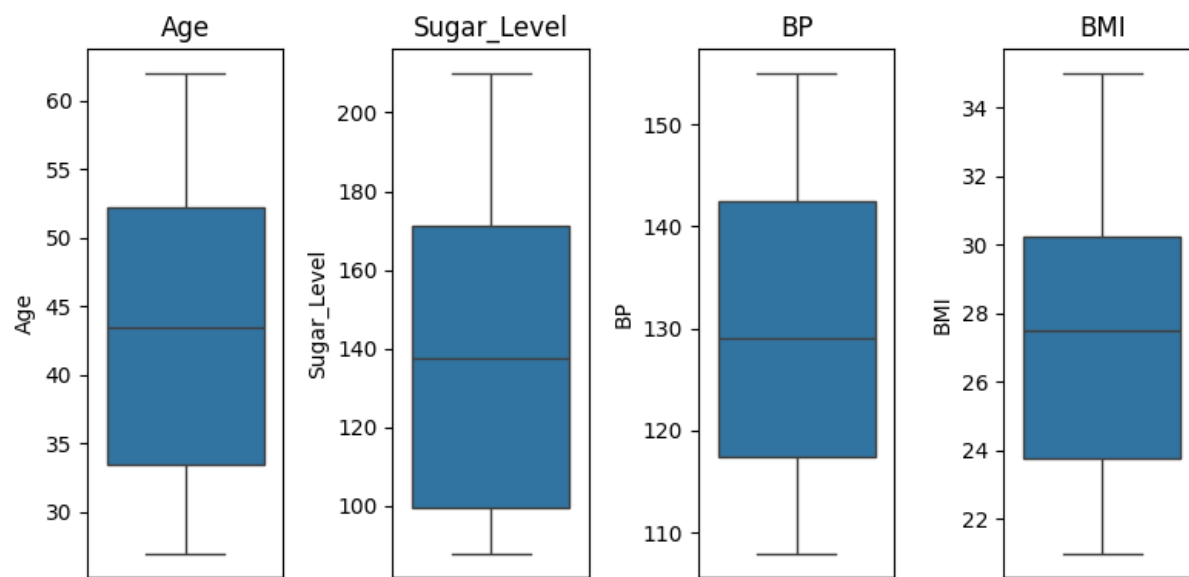
	Age	Sugar_Level	BP	BMI	Disease
Age	1.000000	0.994112	0.994311	0.996335	0.879423
Sugar_Level	0.994112	1.000000	0.993264	0.994264	0.899189
BP	0.994311	0.993264	1.000000	0.993182	0.872432
BMI	0.996335	0.994264	0.993182	1.000000	0.867785
Disease	0.879423	0.899189	0.872432	0.867785	1.000000

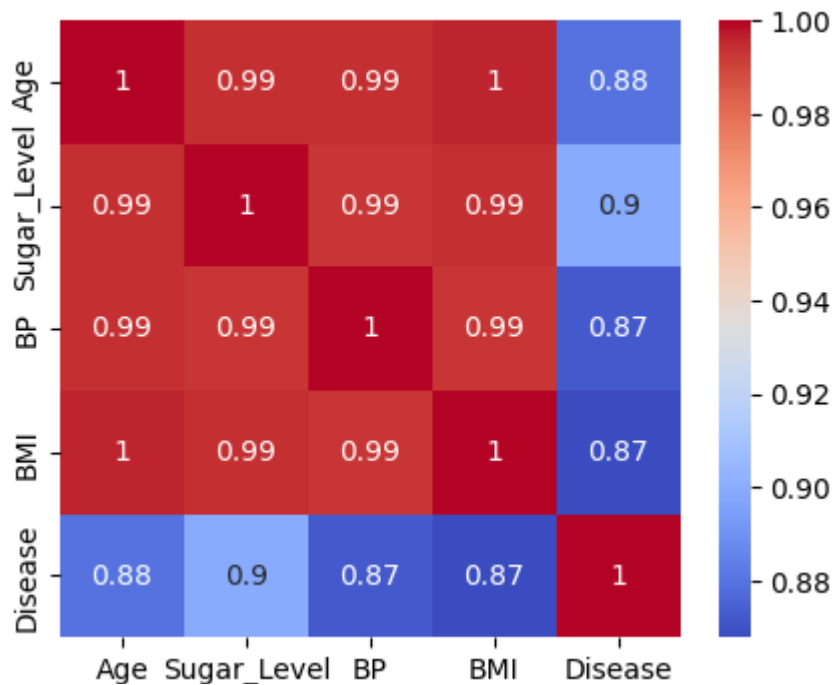
Top 5 Patients

	Patient_ID	Age	Sugar_Level	BP	BMI
10	P11	62	210	155	35
4	P05	60	200	150	34
14	P15	58	190	148	33
2	P03	55	180	145	31
18	P19	53	175	144	31

EDA Completed Successfully.







3. Exploratory Data Analysis (EDA) on the Supermarket Product Sales Dataset.

```
# =====

# Exploratory Data Analysis (EDA)

# Supermarket Product Sales Dataset

# =====

# Import Libraries

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

# Load Dataset

df = pd.read_csv("Product_Sales_Dataset.csv")

# -----

# 1. Display Dataset
```

```
# -----  
  
print("\nFirst Five Records")  
  
print(df.head())  
  
print("\nLast Five Records")  
  
print(df.tail())  
  
# -----  
  
# 2. Dataset Information  
  
# -----  
  
print("\nDataset Information")  
  
print(df.info())  
  
print("\nDataset Shape")  
  
print(df.shape)  
  
print("\nColumn Names")  
  
print(df.columns)  
  
# -----  
  
# 3. Missing Values  
  
# -----  
  
print("\nMissing Values")  
  
print(df.isnull().sum())  
  
# -----  
  
# 4. Duplicate Records  
  
# -----  
  
print("\nDuplicate Records")  
  
print(df.duplicated().sum())  
  
# -----
```



```
# 5. Statistical Summary
```

```
# -----
```

```
print("\nStatistical Summary")
```

```
print(df.describe())
```

```
# -----
```

```
# 6. Category Distribution
```

```
# -----
```

```
print("\nProducts in Each Category")
```

```
print(df["Category"].value_counts())
```

```
# -----
```

```
# 7. Average Values by Category
```

```
# -----
```

```
print("\nAverage Price by Category")
```

```
print(df.groupby("Category")["Price"].mean())
```

```
print("\nAverage Quantity Sold by Category")
```

```
print(df.groupby("Category")["Quantity_Sold"].mean())
```

```
print("\nAverage Discount by Category")
```

```
print(df.groupby("Category")["Discount"].mean())
```

```
print("\nAverage Revenue by Category")
```

```
print(df.groupby("Category")["Revenue"].mean())
```

```
# -----
```

```
# 8. Correlation Matrix
```

```
# -----
```

```
print("\nCorrelation Matrix")
```

```
print(df[["Price", "Quantity_Sold", "Discount", "Revenue"]].corr())
```

```
# -----  
  
# 9. Histograms  
  
# -----  
  
df[["Price","Quantity_Sold","Discount","Revenue"]].hist(figsize=(10,8))  
  
plt.suptitle("Distribution of Numerical Features")  
  
plt.show()  
  
# -----  
  
# 10. Boxplots  
  
# -----  
  
plt.figure(figsize=(12,8))  
  
plt.subplot(2,2,1)  
  
sns.boxplot(y=df["Price"])  
  
plt.title("Price")  
  
plt.subplot(2,2,2)  
  
sns.boxplot(y=df["Quantity_Sold"])  
  
plt.title("Quantity Sold")  
  
plt.subplot(2,2,3)  
  
sns.boxplot(y=df["Discount"])  
  
plt.title("Discount")  
  
plt.subplot(2,2,4)  
  
sns.boxplot(y=df["Revenue"])  
  
plt.title("Revenue")  
  
plt.tight_layout()  
  
plt.show()  
  
# -----
```

```
# 11. Category Count Plot
```

```
# -----
```

```
plt.figure(figsize=(7,5))
```

```
sns.countplot(x="Category", data=df)
```

```
plt.title("Number of Products in Each Category")
```

```
plt.xticks(rotation=20)
```

```
plt.show()
```

```
# -----
```

```
# 12. Revenue by Category
```

```
# -----
```

```
plt.figure(figsize=(7,5))
```

```
sns.barplot(x="Category", y="Revenue", data=df)
```

```
plt.title("Revenue by Category")
```

```
plt.xticks(rotation=20)
```

```
plt.show()
```

```
# -----
```

```
# 13. Price vs Revenue
```

```
# -----
```

```
plt.figure(figsize=(7,5))
```

```
sns.scatterplot(x="Price", y="Revenue", hue="Category", data=df, s=100)
```

```
plt.title("Price vs Revenue")
```

```
plt.show()
```

```
# -----
```

```
# 14. Quantity Sold vs Revenue
```

```
# -----
```

```

plt.figure(figsize=(7,5))

sns.scatterplot(x="Quantity_Sold", y="Revenue", hue="Category", data=df, s=100)

plt.title("Quantity Sold vs Revenue")

plt.show()

# -----

# 15. Discount vs Revenue

# -----

plt.figure(figsize=(7,5))

sns.scatterplot(x="Discount", y="Revenue", hue="Category", data=df, s=100)

plt.title("Discount vs Revenue")

plt.show()

# -----

# 16. Pair Plot

# -----

sns.pairplot(df, hue="Category")

plt.show()

# -----

# 17. Correlation Heatmap

# -----

plt.figure(figsize=(8,6))

sns.heatmap(df[["Price", "Quantity_Sold", "Discount", "Revenue"]].corr(),

annot=True,

cmap="coolwarm",

linewidths=0.5)

plt.title("Correlation Heatmap")

```

```

plt.show()

# -----

# 18. Top 5 High-Selling Products

# -----

print("\nTop 5 High-Selling Products")

top_sales = df.sort_values(by="Quantity_Sold", ascending=False)

print(top_sales[["Product_ID", "Category", "Quantity_Sold"]].head())

# -----

# 19. Top 5 Revenue-Generating Products

# -----

print("\nTop 5 Revenue Generating Products")

top_revenue = df.sort_values(by="Revenue", ascending=False)

print(top_revenue[["Product_ID", "Category", "Revenue"]].head())

# -----

# 20. EDA Summary

# -----

print("\n=====")

print("EDA SUMMARY")

print("=====")

print("Total Products :", len(df))

print("\nHighest Selling Product")

print(df.loc[df["Quantity_Sold"].idxmax()][["Product_ID", "Quantity_Sold"]])

print("\nHighest Revenue Product")

print(df.loc[df["Revenue"].idxmax()][["Product_ID", "Revenue"]])

print("\nAverage Revenue by Category")

```

```
print(df.groupby("Category")["Revenue"].mean())

print("\nAverage Quantity Sold by Category")

print(df.groupby("Category")["Quantity_Sold"].mean())

print("\nEDA Completed Successfully.")
```

Output:

First Five Records

	Product_ID	Category	Price	Quantity_Sold	Discount	Revenue
0	PR01	Grocery	50	20	5	1000
1	PR02	Snacks	30	35	3	1050
2	PR03	Dairy	45	18	2	810
3	PR04	Fruits	60	25	5	1500
4	PR05	Vegetables	40	30	4	1200

Last Five Records

	Product_ID	Category	Price	Quantity_Sold	Discount	Revenue
15	PR16	Grocery	100	10	7	1000
16	PR17	Snacks	35	32	3	1120
17	PR18	Dairy	50	19	2	950
18	PR19	Fruits	80	18	6	1440
19	PR20	Vegetables	30	35	3	1050

Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Product_ID      20 non-null    object
1   Category        20 non-null    object
2   Price           20 non-null    int64
3   Quantity_Sold   20 non-null    int64
4   Discount        20 non-null    int64
5   Revenue         20 non-null    int64
dtypes: int64(4), object(2)
memory usage: 1.1+ KB
None
```

Dataset Shape

```
(20, 6)
```

Column Names

```
Index(['Product_ID', 'Category', 'Price', 'Quantity_Sold', 'Discount',
      'Revenue'],
      dtype='object')
```

Missing Values

```
Product_ID      0
Category        0
Price           0
Quantity_Sold    0
Discount        0
Revenue         0
dtype: int64
```

Duplicate Records
0

Statistical Summary

	Price	Quantity_Sold	Discount	Revenue
count	20.000000	20.000000	20.000000	20.000000
mean	54.000000	24.000000	4.000000	1114.000000
std	22.803509	9.673621	1.555973	221.416493
min	20.000000	10.000000	2.000000	810.000000
25%	35.000000	17.500000	3.000000	972.500000
50%	50.000000	21.000000	4.000000	1050.000000
75%	71.250000	30.500000	5.000000	1200.000000
max	100.000000	45.000000	7.000000	1540.000000

Products in Each Category

Category

Grocery	4
Snacks	4
Dairy	4
Fruits	4
Vegetables	4

Name: count, dtype: int64

Average Price by Category

Category

Dairy	53.75
Fruits	71.25
Grocery	80.00
Snacks	27.50
Vegetables	37.50

Name: Price, dtype: float64

Average Quantity Sold by Category

Category

Dairy	16.75
Fruits	21.25
Grocery	14.25
Snacks	38.00
Vegetables	29.75

Name: Quantity_Sold, dtype: float64

Average Discount by Category

Category

Dairy	2.25
Fruits	5.25
Grocery	6.00
Snacks	2.75
Vegetables	3.75

Name: Discount, dtype: float64

Average Revenue by Category

Category

Dairy	887.5
Fruits	1495.0
Grocery	1070.0
Snacks	1017.5
Vegetables	1100.0

Name: Revenue, dtype: float64

Correlation Matrix

	Price	Quantity_Sold	Discount	Revenue
--	-------	---------------	----------	---------

Price	1.000000	-0.868474	0.786174	0.388086
Quantity_Sold	-0.868474	1.000000	-0.507018	-0.081335
Discount	0.786174	-0.507018	1.000000	0.575938
Revenue	0.388086	-0.081335	0.575938	1.000000

Top 5 High-Selling Products

	Product_ID	Category	Quantity_Sold
11	PR12	Snacks	45
6	PR07	Snacks	40
1	PR02	Snacks	35
19	PR20	Vegetables	35
16	PR17	Snacks	32

Top 5 Revenue Generating Products

	Product_ID	Category	Revenue
8	PR09	Fruits	1540
3	PR04	Fruits	1500
13	PR14	Fruits	1500
18	PR19	Fruits	1440
5	PR06	Grocery	1200

=====

EDA SUMMARY

=====

Total Products : 20

Highest Selling Product

Product_ID	PR12
Quantity_Sold	45

Name: 11, dtype: object

Highest Revenue Product

Product_ID	PR09
Revenue	1540

Name: 8, dtype: object

Average Revenue by Category

Category	
Dairy	887.5
Fruits	1495.0
Grocery	1070.0
Snacks	1017.5
Vegetables	1100.0

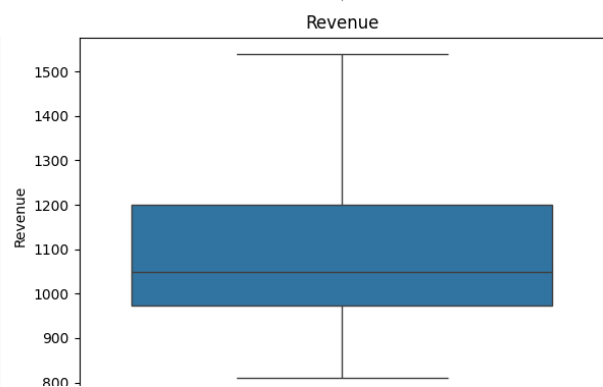
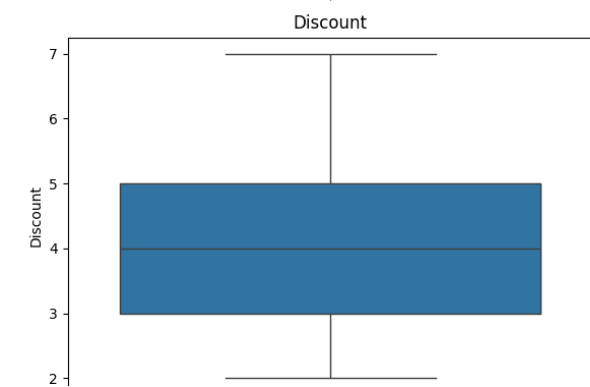
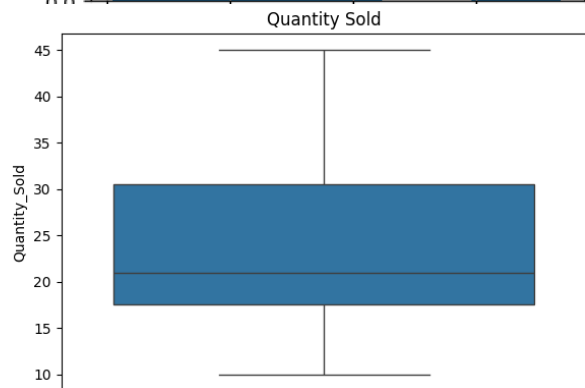
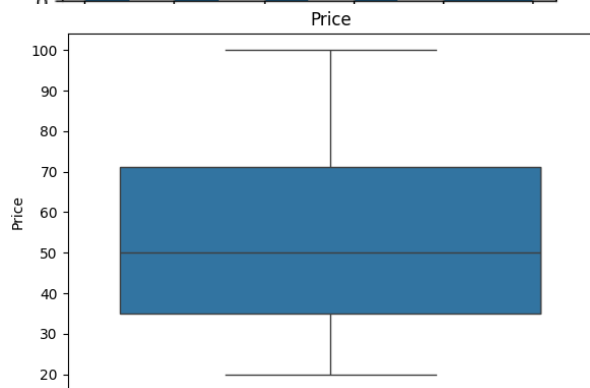
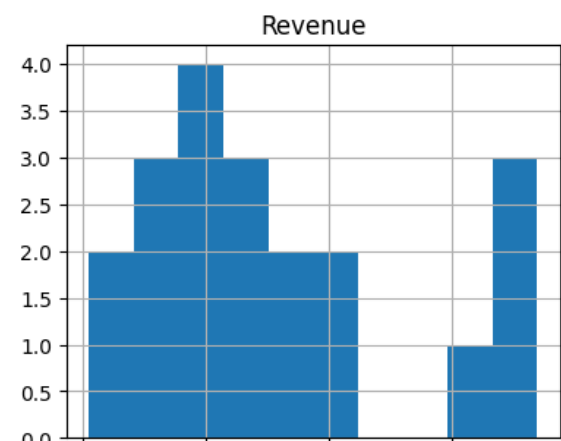
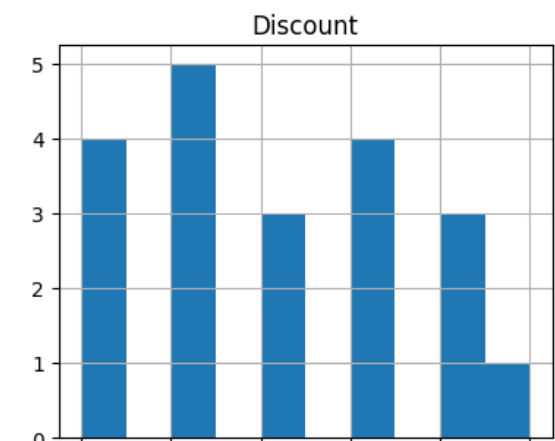
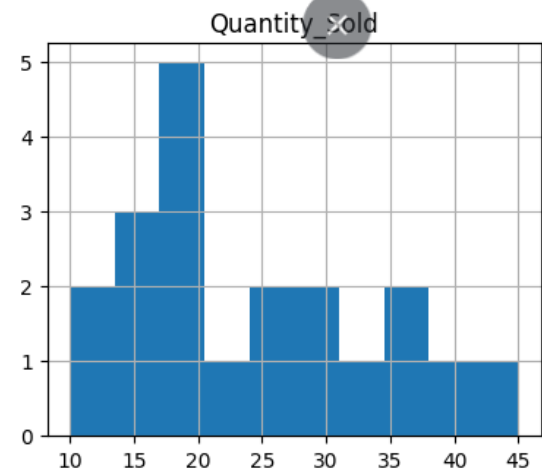
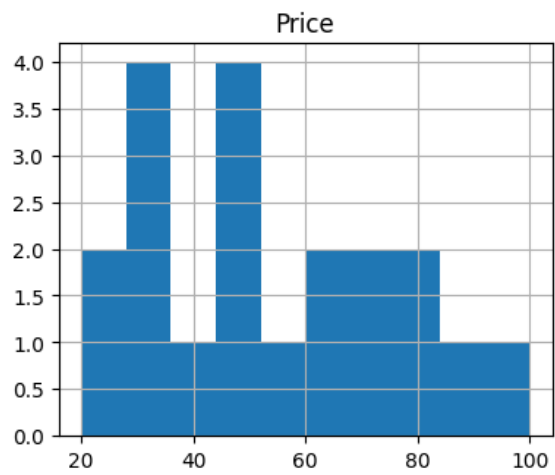
Name: Revenue, dtype: float64

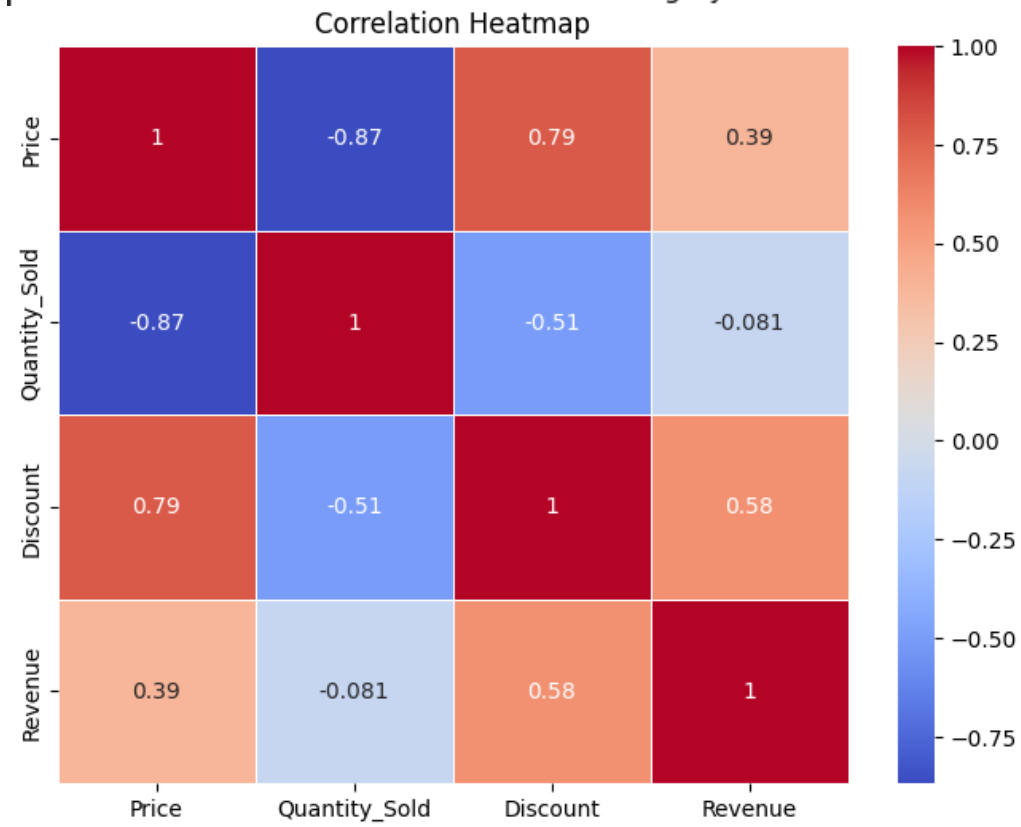
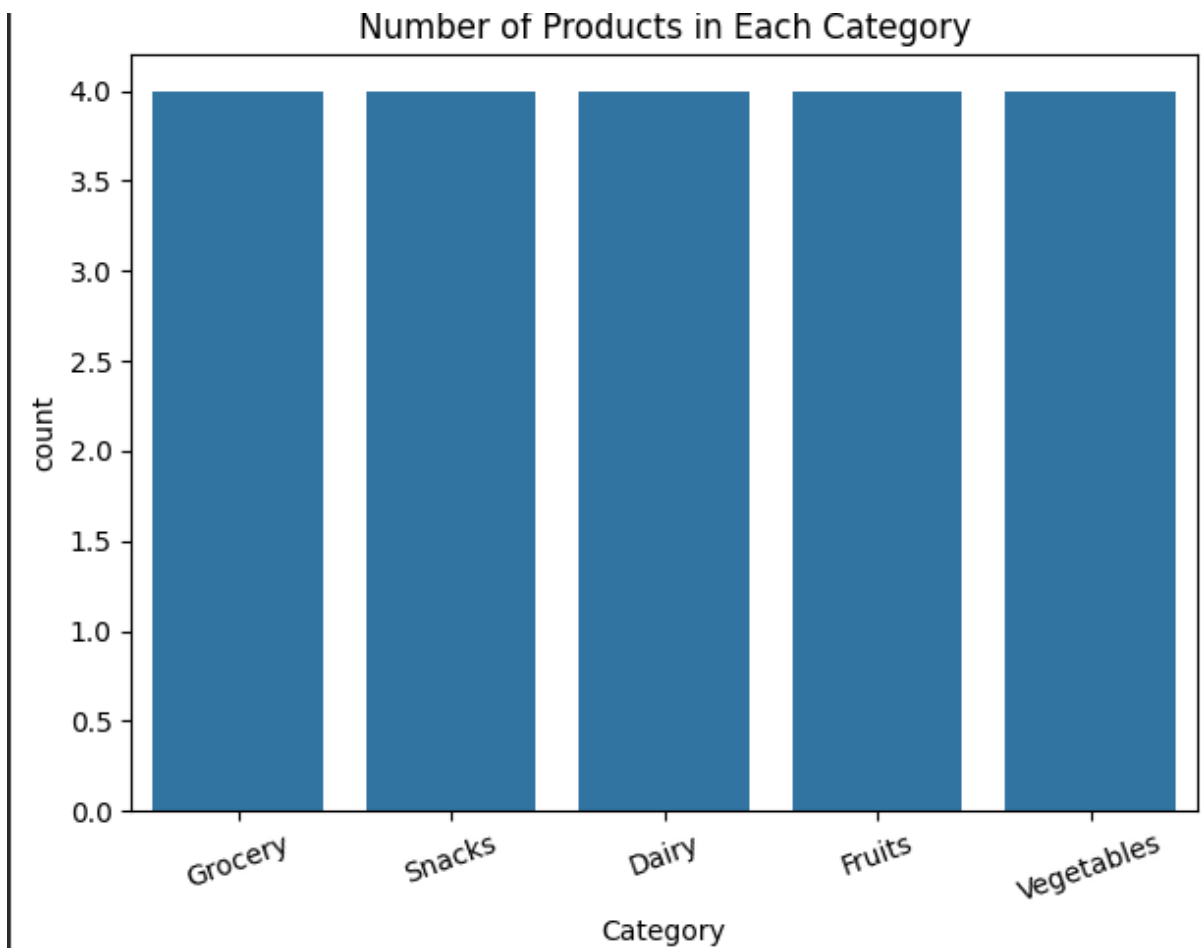
Average Quantity Sold by Category

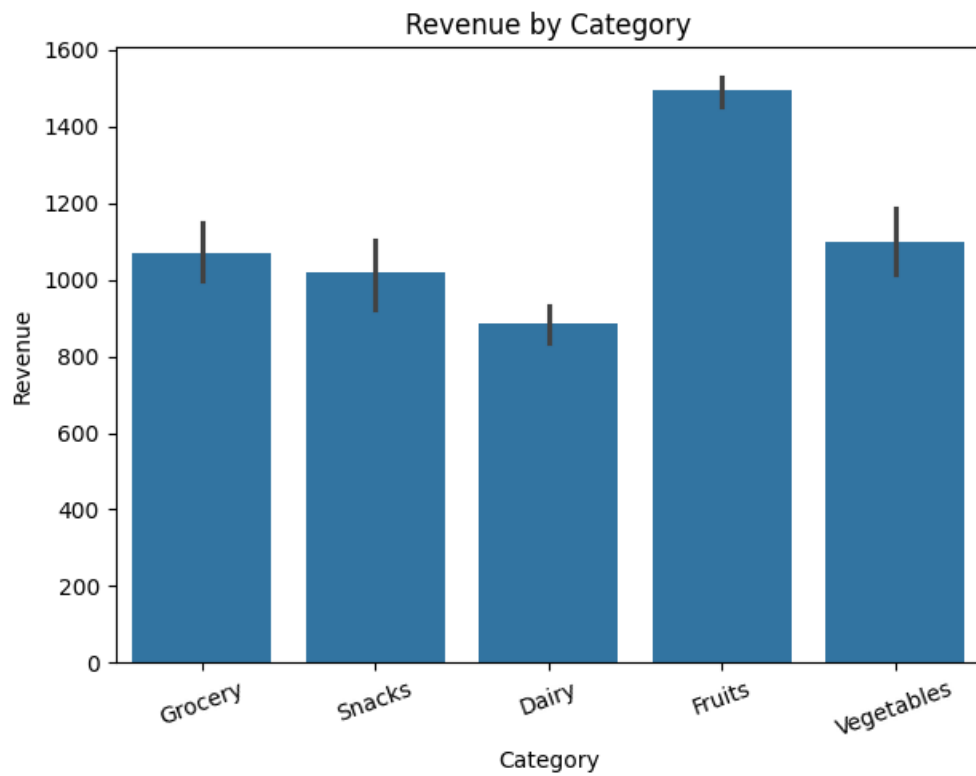
Category	
Dairy	16.75
Fruits	21.25
Grocery	14.25
Snacks	38.00
Vegetables	29.75

Name: Quantity_Sold, dtype: float64

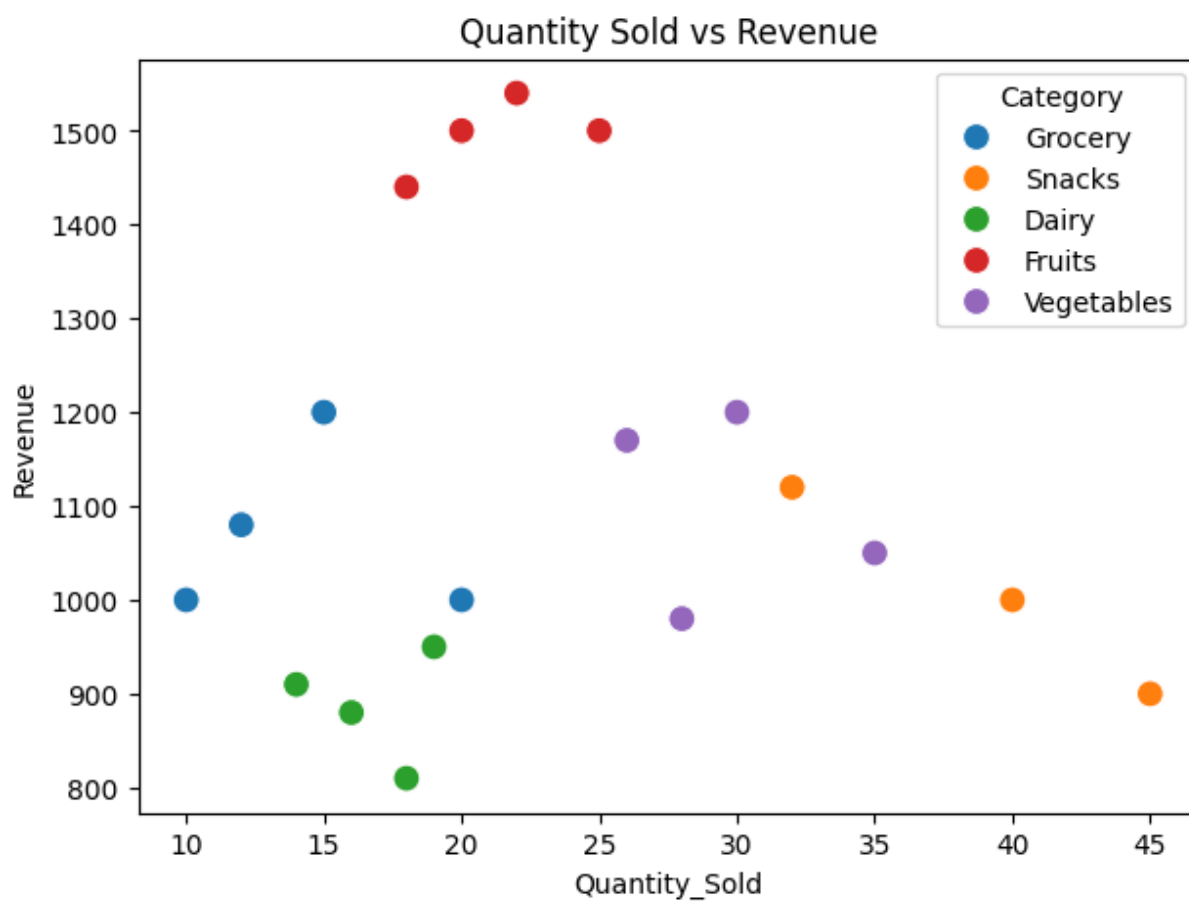
EDA Completed Successfully.

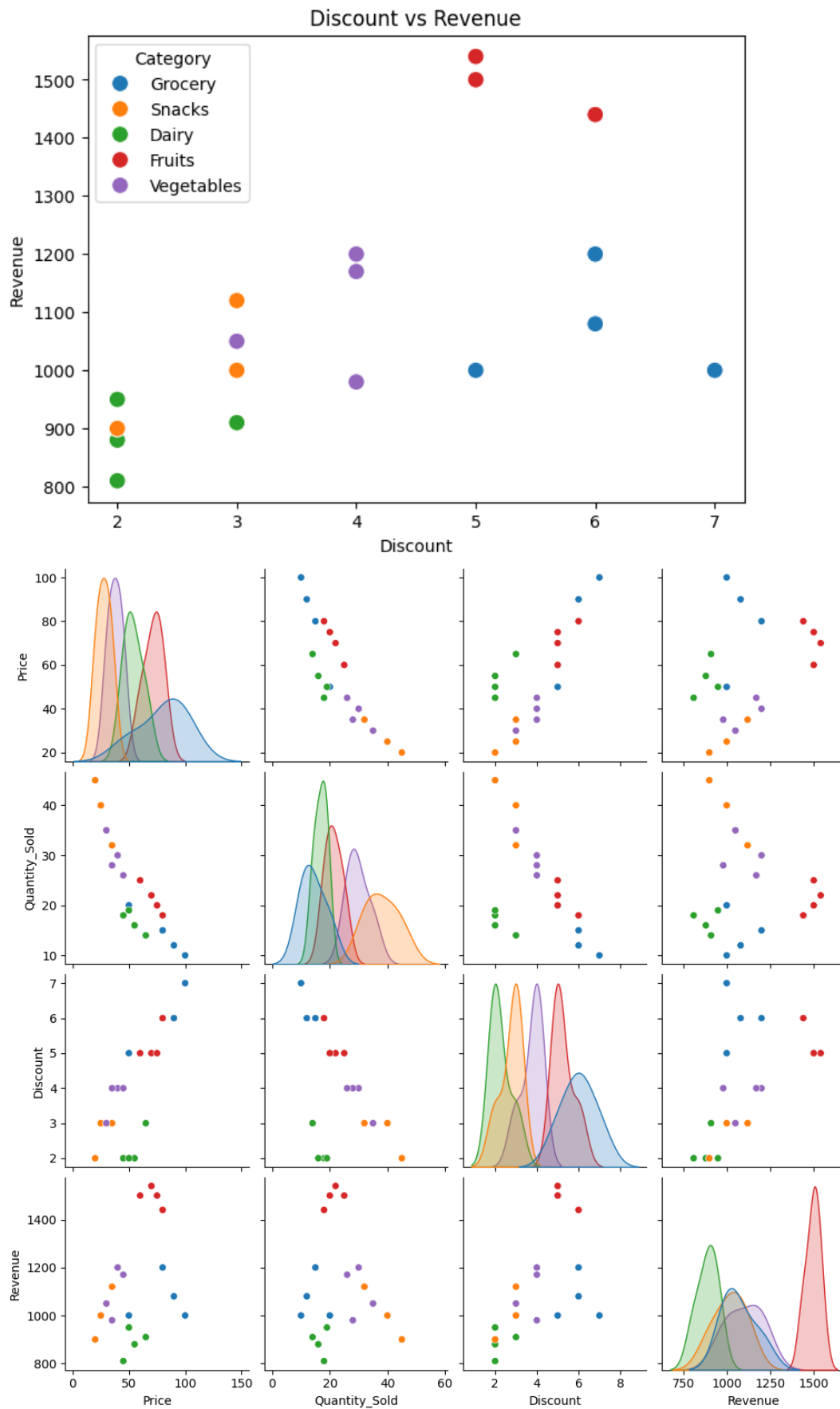






Price vs Revenue





4. Exploratory Data Analysis (EDA) on the Loan Approval Dataset.

```
# =====  
  
# Exploratory Data Analysis (EDA)  
  
# Loan Approval Dataset  
  
# =====  
  
# Import Libraries  
  
import pandas as pd  
  
import matplotlib.pyplot as plt  
  
import seaborn as sns  
  
# Load Dataset  
  
df = pd.read_csv("Loan_Approval_Dataset.csv")  
  
# -----  
  
# 1. Display Dataset  
  
# -----  
  
print("\nFirst Five Records")  
  
print(df.head())  
  
print("\nLast Five Records")  
  
print(df.tail())  
  
# -----  
  
# 2. Dataset Information  
  
# -----  
  
print("\nDataset Information")  
  
print(df.info())  
  
print("\nDataset Shape")  
  
print(df.shape)
```

```

print("\nColumn Names")

print(df.columns)

# -----

# 3. Missing Values

# -----

print("\nMissing Values")

print(df.isnull().sum())

# -----

# 4. Duplicate Records

# -----

print("\nDuplicate Records")

print(df.duplicated().sum())

# -----

# 5. Statistical Summary

# -----

print("\nStatistical Summary")

print(df.describe())

# -----

# 6. Loan Status Distribution

# -----

print("\nLoan Status Count")

print(df["Loan_Status"].value_counts())

print("\nLoan Status Percentage")

print(df["Loan_Status"].value_counts(normalize=True) * 100)

# -----

```

```
# 7. Employment Type Distribution
```

```
# -----
```

```
print("\nEmployment Type Count")
```

```
print(df["Employment_Type"].value_counts())
```

```
# -----
```

```
# 8. Average Values by Loan Status
```

```
# -----
```

```
print("\nAverage Values by Loan Status")
```

```
print(df.groupby("Loan_Status")[["Income", "Credit_Score", "Loan_Amount"]].mean())
```

```
# -----
```

```
# 9. Correlation Matrix
```

```
# -----
```

```
# Convert Loan Status to Numeric
```

```
df["Loan"] = df["Loan_Status"].map({"Rejected":0, "Approved":1})
```

```
print("\nCorrelation Matrix")
```

```
print(df[["Income", "Credit_Score", "Loan_Amount", "Loan"]].corr())
```

```
# -----
```

```
# 10. Histograms
```

```
# -----
```

```
df[["Income", "Credit_Score", "Loan_Amount"]].hist(figsize=(10,8))
```

```
plt.suptitle("Distribution of Numerical Features")
```

```
plt.show()
```

```
# -----
```

```
# 11. Boxplots
```

```
# -----
```

```
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)

sns.boxplot(y=df["Income"])

plt.title("Income")

plt.subplot(1,3,2)

sns.boxplot(y=df["Credit_Score"])

plt.title("Credit Score")

plt.subplot(1,3,3)

sns.boxplot(y=df["Loan_Amount"])

plt.title("Loan Amount")

plt.tight_layout()

plt.show()

# -----

# 12. Loan Status Count Plot

# -----

plt.figure(figsize=(5,4))

sns.countplot(x="Loan_Status", data=df)

plt.title("Loan Approval Distribution")

plt.show()

# -----

# 13. Employment Type Count Plot

# -----

plt.figure(figsize=(6,4))

sns.countplot(x="Employment_Type", data=df)

plt.title("Employment Type Distribution")
```



```
plt.xticks(rotation=15)

plt.show()

# -----

# 14. Income vs Loan Status

# -----

plt.figure(figsize=(6,4))

sns.boxplot(x="Loan_Status", y="Income", data=df)

plt.title("Income vs Loan Status")

plt.show()

# -----

# 15. Credit Score vs Loan Status

# -----

plt.figure(figsize=(6,4))

sns.boxplot(x="Loan_Status", y="Credit_Score", data=df)

plt.title("Credit Score vs Loan Status")

plt.show()

# -----

# 16. Loan Amount vs Loan Status

# -----

plt.figure(figsize=(6,4))

sns.boxplot(x="Loan_Status", y="Loan_Amount", data=df)

plt.title("Loan Amount vs Loan Status")

plt.show()

# -----

# 17. Income vs Credit Score
```

```

# -----

plt.figure(figsize=(6,5))

sns.scatterplot(x="Income",
y="Credit_Score",
hue="Loan_Status",
style="Employment_Type",
s=100,
data=df)

plt.title("Income vs Credit Score")

plt.show()

# -----

# 18. Pair Plot

# -----

sns.pairplot(df,
vars=["Income","Credit_Score","Loan_Amount"],
hue="Loan_Status")

plt.show()

# -----

# 19. Correlation Heatmap

# -----

plt.figure(figsize=(7,5))

sns.heatmap(df[["Income",
"Credit_Score",
"Loan_Amount",
"Loan"]].corr(),

```

```

annot=True,

cmap="coolwarm",

linewidths=0.5)

plt.title("Correlation Heatmap")

plt.show()

# -----

# 20. Top Approved Customers

# -----

print("\nApproved Customers")

print(df[df["Loan_Status"]=="Approved"][["Customer_ID",

"Income",

"Credit_Score",

"Loan_Amount"]])

# -----

# 21. Rejected Customers

# -----

print("\nRejected Customers")

print(df[df["Loan_Status"]=="Rejected"][["Customer_ID",

"Income",

"Credit_Score",

"Loan_Amount"]])

# -----

# 22. EDA Summary

# -----

print("\n=====")

```

```

print("EDA SUMMARY")

print("=====")

print("Total Customers :", len(df))

print("\nApproved Loans :", (df["Loan_Status"]=="Approved").sum())

print("Rejected Loans :", (df["Loan_Status"]=="Rejected").sum())

print("\nAverage Income by Loan Status")

print(df.groupby("Loan_Status")["Income"].mean())

print("\nAverage Credit Score by Loan Status")

print(df.groupby("Loan_Status")["Credit_Score"].mean())

print("\nAverage Loan Amount by Loan Status")

print(df.groupby("Loan_Status")["Loan_Amount"].mean())

print("\nEmployment Type vs Loan Status")

print(pd.crosstab(df["Employment_Type"], df["Loan_Status"]))

print("\nEDA Completed Successfully.")

```

Output:

First Five Records

	Customer_ID	Income	Credit_Score	Loan_Amount	Employment_Type	Loan_Status
0	C01	45000	720	200000	Salaried	Approved
1	C02	25000	580	150000	Self-employed	Rejected
2	C03	60000	750	300000	Salaried	Approved
3	C04	22000	560	120000	Unemployed	Rejected
4	C05	50000	710	250000	Salaried	Approved

Last Five Records

	Customer_ID	Income	Credit_Score	Loan_Amount	Employment_Type	Loan_Status
15	C16	32000	610	190000	Self-employed	Rejected
16	C17	52000	725	260000	Salaried	Approved
17	C18	27000	575	150000	Self-employed	Rejected
18	C19	68000	770	330000	Salaried	Approved
19	C20	23000	555	125000	Unemployed	Rejected

Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
```

```

RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer_ID           20 non-null    object
1   Income                 20 non-null    int64
2   Credit_Score           20 non-null    int64
3   Loan_Amount            20 non-null    int64
4   Employment_Type        20 non-null    object
5   Loan_Status            20 non-null    object
dtypes: int64(3), object(3)
memory usage: 1.1+ KB
None

```

```

Dataset Shape
(20, 6)

```

```

Column Names
Index(['Customer_ID', 'Income', 'Credit_Score', 'Loan_Amount',
      'Employment_Type', 'Loan_Status'],
      dtype='object')

```

```

Missing Values
Customer_ID      0
Income           0
Credit_Score     0
Loan_Amount      0
Employment_Type  0
Loan_Status      0
dtype: int64

```

```

Duplicate Records
0

```

```

Statistical Summary
              Income  Credit_Score  Loan_Amount
count      20.000000      20.000000      20.000000
mean    43000.000000      664.000000    223250.000000
std    17923.815383      85.526235     83733.207272
min     22000.000000     555.000000    120000.000000
25%     26750.000000     583.750000    150000.000000
50%     40000.000000     660.000000    200000.000000
75%     56250.000000     735.000000    277500.000000
max     75000.000000     790.000000    400000.000000

```

```

Loan Status Count
Loan_Status
Approved      10
Rejected      10
Name: count, dtype: int64

```

```

Loan Status Percentage
Loan_Status
Approved      50.0
Rejected      50.0
Name: proportion, dtype: float64

```

```

Employment Type Count
Employment_Type
Salaried      10
Self-employed   6

```

Unemployed 4
Name: count, dtype: int64

Average Values by Loan Status

	Income	Credit_Score	Loan_Amount
Loan_Status			
Approved	58800.0	743.5	292000.0
Rejected	27200.0	584.5	154500.0

Correlation Matrix

	Income	Credit_Score	Loan_Amount	Loan
Income	1.000000	0.983995	0.987882	0.904409
Credit_Score	0.983995	1.000000	0.953689	0.953687
Loan_Amount	0.987882	0.953689	1.000000	0.842390
Loan	0.904409	0.953687	0.842390	1.000000

Approved Customers

	Customer_ID	Income	Credit_Score	Loan_Amount
0	C01	45000	720	200000
2	C03	60000	750	300000
4	C05	50000	710	250000
6	C07	70000	780	350000
8	C09	55000	730	270000
10	C11	65000	760	320000
12	C13	48000	700	240000
14	C15	75000	790	400000
16	C17	52000	725	260000
18	C19	68000	770	330000

Rejected Customers

	Customer_ID	Income	Credit_Score	Loan_Amount
1	C02	25000	580	150000
3	C04	22000	560	120000
5	C06	30000	600	180000
7	C08	28000	590	160000
9	C10	24000	570	130000
11	C12	35000	620	200000
13	C14	26000	585	140000
15	C16	32000	610	190000
17	C18	27000	575	150000
19	C20	23000	555	125000

EDA SUMMARY

Total Customers : 20

Approved Loans : 10

Rejected Loans : 10

Average Income by Loan Status

Loan_Status	Income
Approved	58800.0
Rejected	27200.0

Name: Income, dtype: float64

Average Credit Score by Loan Status

Loan_Status	Credit_Score
Approved	743.5
Rejected	584.5

Name: Credit_Score, dtype: float64

Average Loan Amount by Loan Status

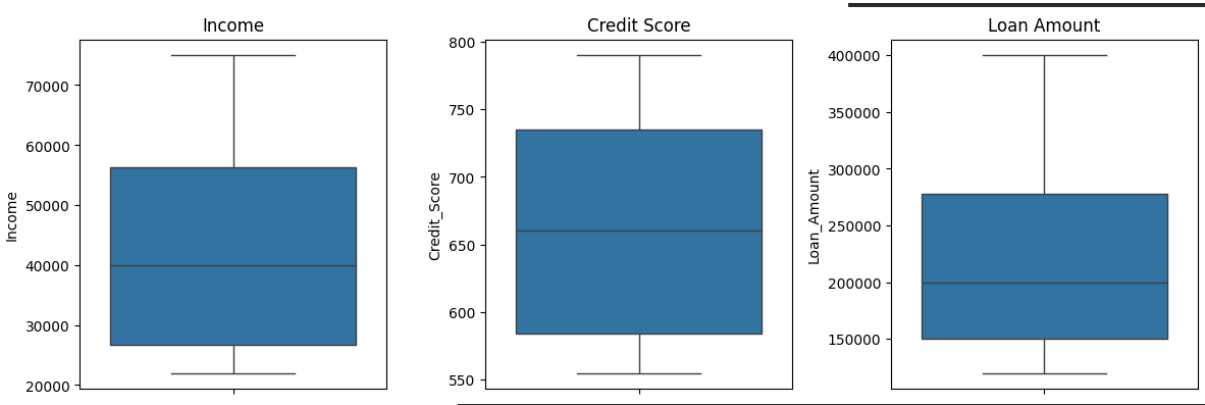
Loan_Status	
Approved	292000.0
Rejected	154500.0

Name: Loan_Amount, dtype: float64

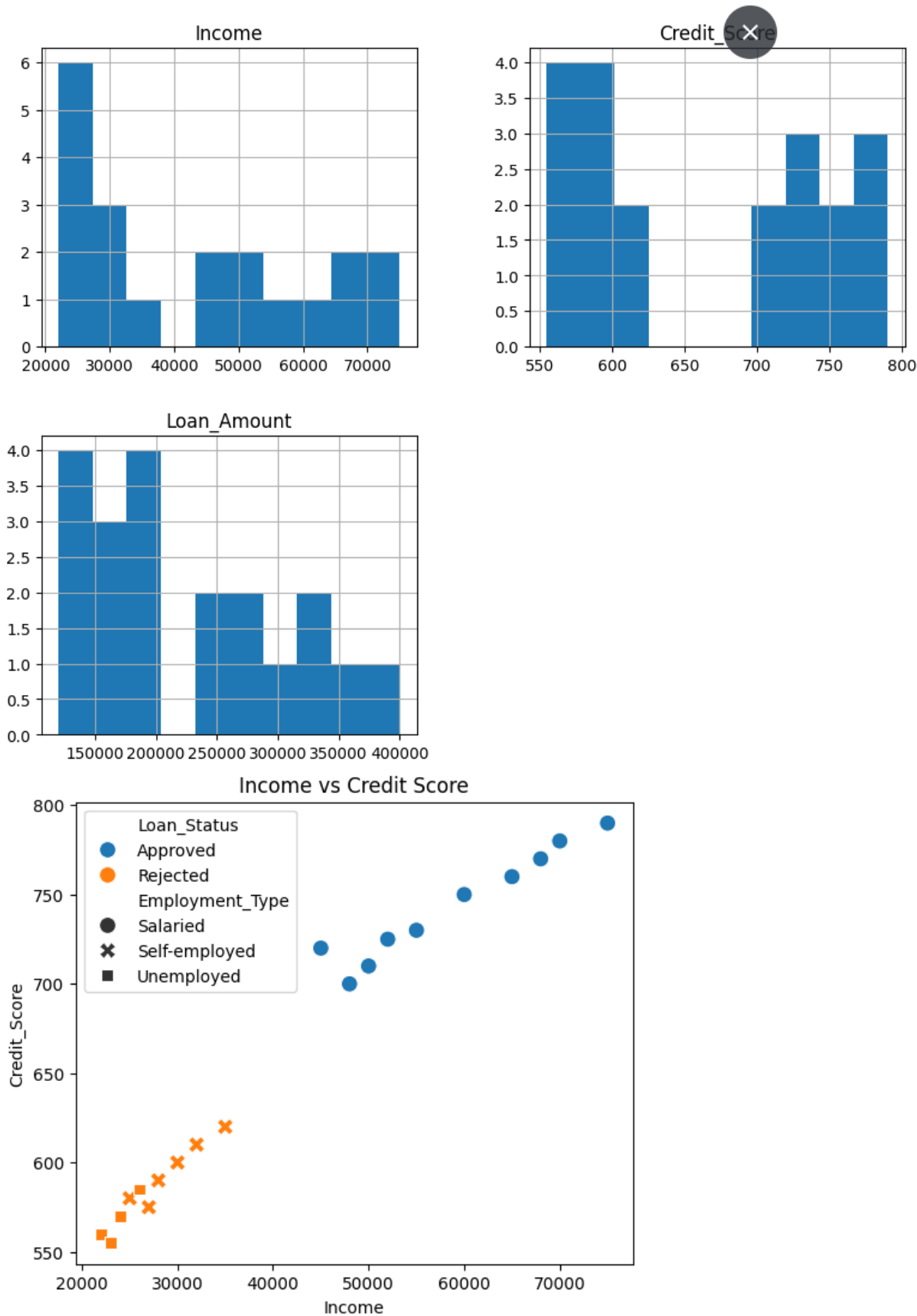
Employment Type vs Loan Status

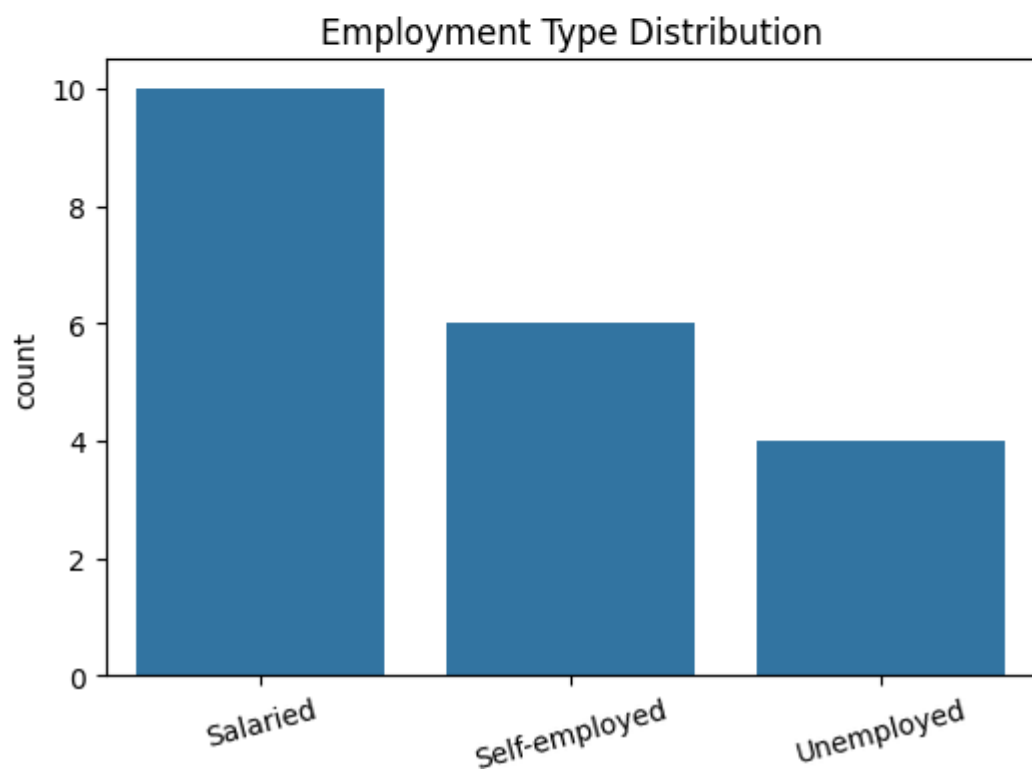
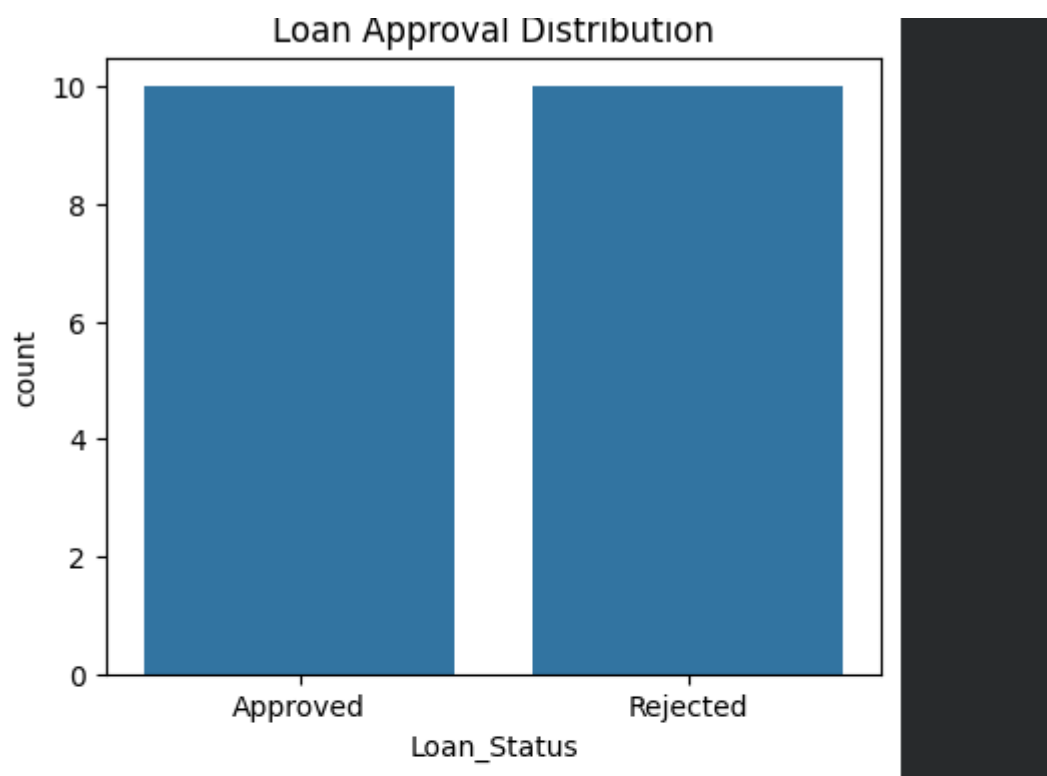
Loan_Status	Approved	Rejected
Employment_Type		
Salaried	10	0
Self-employed	0	6
Unemployed	0	4

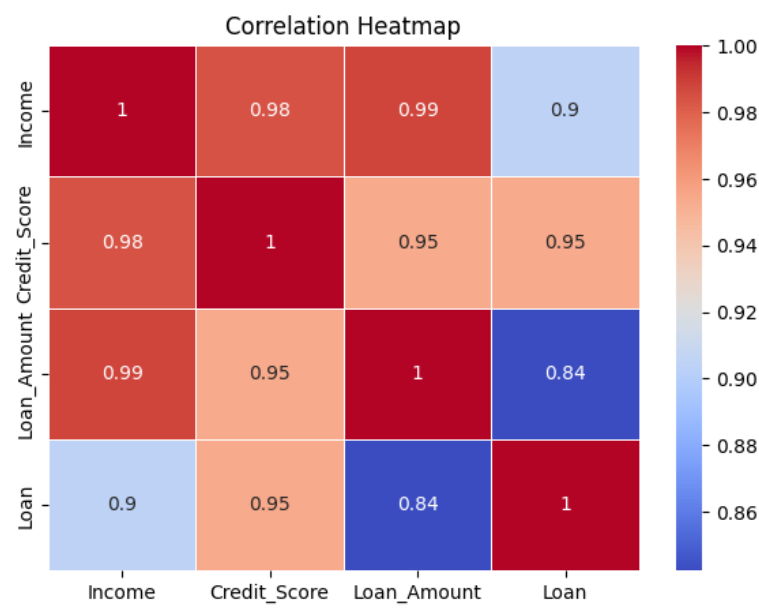
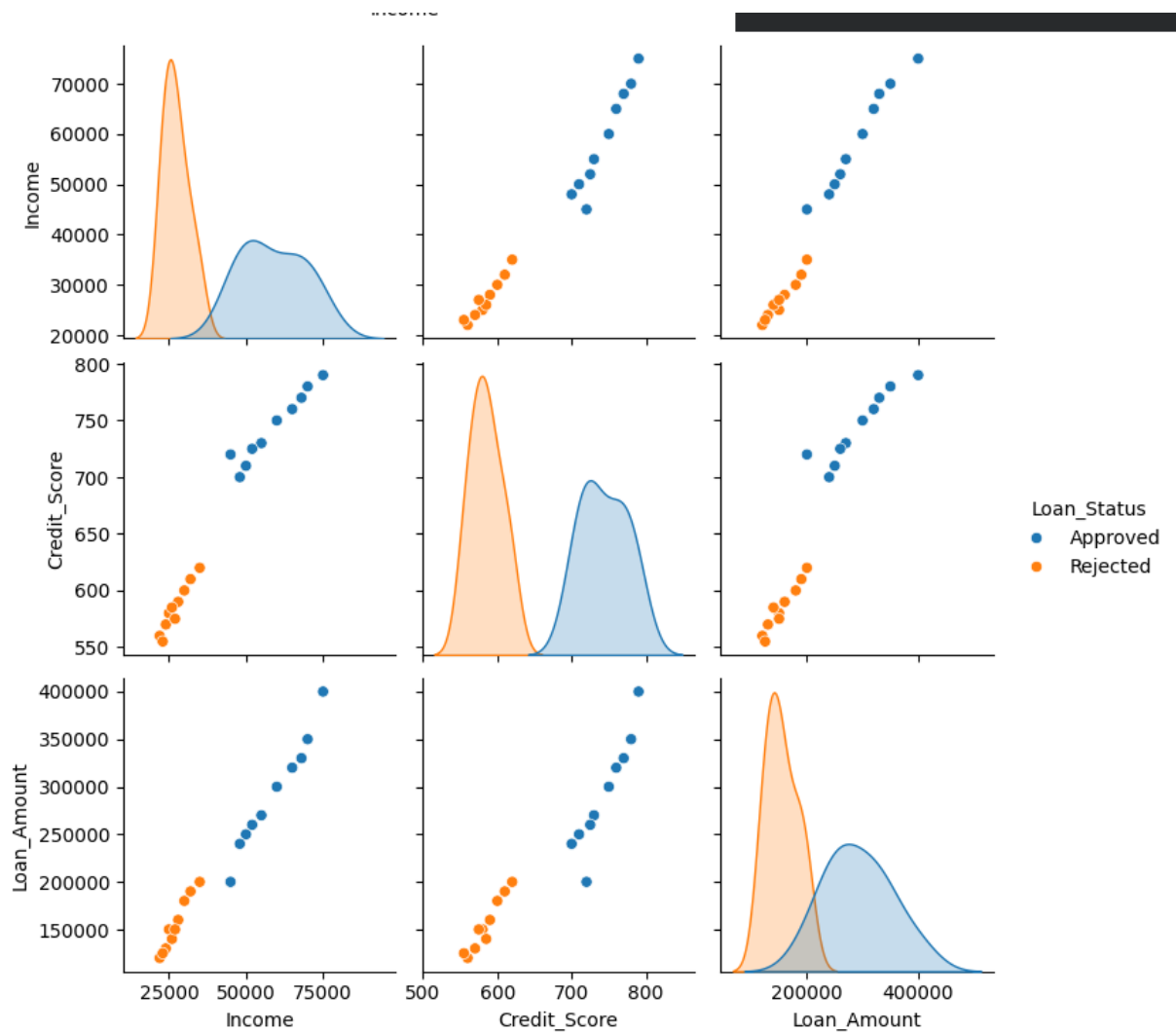
EDA Completed Successfully.



Distribution of Numerical Features







5. Exploratory Data Analysis (EDA) of Crop Yield Dataset

```
# Exploratory Data Analysis (EDA)

# Crop Yield Dataset

# Import Libraries

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

# Load Dataset

df = pd.read_csv("Farm_Crop_Yield_Dataset.csv")

# 1. Display Dataset

print("\nFirst Five Records")

print(df.head())

print("\nLast Five Records")

print(df.tail())


# 2. Dataset Information

print("\nDataset Information")

print(df.info())

print("\nDataset Shape")

print(df.shape)

print("\nColumn Names")

print(df.columns)


# 3. Missing Values

print("\nMissing Values")
```

```

print(df.isnull().sum())

# -----

# 4. Duplicate Records

# -----

print("\nDuplicate Records")

print(df.duplicated().sum())

# -----

# 5. Statistical Summary

# -----

print("\nStatistical Summary")

print(df.describe())

# Mean

print("\nMean")

print(df[["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]].mean())

# Median

print("\nMedian")

print(df[["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]].median())

# Minimum

print("\nMinimum")

print(df[["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]].min())

# Maximum

print("\nMaximum")

print(df[["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]].max())

# Standard Deviation

print("\nStandard Deviation")

```

```
print(df[["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]].std())
```

```
# -----
```

```
# 6. Soil Type Distribution
```

```
# -----
```

```
print("\nSoil Type Count")
```

```
print(df["Soil_Type"].value_counts())
```

```
# -----
```

```
# 7. Average Crop Yield by Soil Type
```

```
# -----
```

```
print("\nAverage Crop Yield by Soil Type")
```

```
print(df.groupby("Soil_Type")["Crop_Yield_kg"].mean())
```

```
# -----
```

```
# 8. Correlation Matrix
```

```
# -----
```

```
print("\nCorrelation Matrix")
```

```
print(df[["Rainfall_mm",
```

```
"Temperature",
```

```
"Fertilizer_kg",
```

```
"Crop_Yield_kg"]].corr())
```

```
# -----
```

```
# 9. Bar Chart
```

```
# -----
```

```
plt.figure(figsize=(7,5))
```

```
sns.barplot(x="Soil_Type",
```

```
y="Crop_Yield_kg",
```

```
data=df)

plt.title("Average Crop Yield by Soil Type")

plt.xlabel("Soil Type")

plt.ylabel("Crop Yield (kg)")

plt.show()

# -----

# 10. Histograms

# -----

df[["Rainfall_mm",

"Temperature",

"Fertilizer_kg",

"Crop_Yield_kg"]].hist(figsize=(10,8))

plt.suptitle("Distribution of Numerical Features")

plt.show()

# -----

# 11. Scatter Plot

# -----

# Rainfall vs Crop Yield

plt.figure(figsize=(6,4))

sns.scatterplot(x="Rainfall_mm",

y="Crop_Yield_kg",

hue="Soil_Type",

data=df,

s=100)

plt.title("Rainfall vs Crop Yield")
```

```

plt.show()

# Temperature vs Crop Yield

plt.figure(figsize=(6,4))

sns.scatterplot(x="Temperature",
y="Crop_Yield_kg",
hue="Soil_Type",
data=df,
s=100)

plt.title("Temperature vs Crop Yield")

plt.show()

# Fertilizer vs Crop Yield

plt.figure(figsize=(6,4))

sns.scatterplot(x="Fertilizer_kg",
y="Crop_Yield_kg",
hue="Soil_Type",
data=df,
s=100)

plt.title("Fertilizer vs Crop Yield")

plt.show()

# -----

# 12. Boxplots

# -----

plt.figure(figsize=(12,8))

plt.subplot(2,2,1)

sns.boxplot(y=df["Rainfall_mm"])

```

```
plt.title("Rainfall")

plt.subplot(2,2,2)

sns.boxplot(y=df["Temperature"])

plt.title("Temperature")

plt.subplot(2,2,3)

sns.boxplot(y=df["Fertilizer_kg"])

plt.title("Fertilizer")

plt.subplot(2,2,4)

sns.boxplot(y=df["Crop_Yield_kg"])

plt.title("Crop Yield")

plt.tight_layout()

plt.show()

# -----

# 13. Pair Plot

# -----

sns.pairplot(df,

vars=["Rainfall_mm",

"Temperature",

"Fertilizer_kg",

"Crop_Yield_kg"],

hue="Soil_Type")

plt.show()

# -----

# 14. Correlation Heatmap

# -----
```



```

plt.figure(figsize=(7,5))

sns.heatmap(df[["Rainfall_mm",

"Temperature",

"Fertilizer_kg",

"Crop_Yield_kg"]].corr(),

annot=True,

cmap="coolwarm",

linewidths=0.5)

plt.title("Correlation Heatmap")

plt.show()

# -----

# 15. Top 5 Highest Yield Farms

# -----

print("\nTop 5 Highest Yield Farms")

top_yield = df.sort_values(by="Crop_Yield_kg",

ascending=False)

print(top_yield[["Farm_ID",

"Soil_Type",

"Crop_Yield_kg"]].head())

# -----

# 16. EDA Summary

# -----

print("\n=====")

print("EDA SUMMARY")

print("=====")

```

```

print("Total Farms :", len(df))

print("\nHighest Crop Yield")

print(df.loc[df["Crop_Yield_kg"].idxmax()][["Farm_ID",
"Crop_Yield_kg"]])

print("\nLowest Crop Yield")

print(df.loc[df["Crop_Yield_kg"].idxmin()][["Farm_ID",
"Crop_Yield_kg"]])

print("\nAverage Crop Yield by Soil Type")

print(df.groupby("Soil_Type")["Crop_Yield_kg"].mean())

print("\nAverage Rainfall by Soil Type")

print(df.groupby("Soil_Type")["Rainfall_mm"].mean())

print("\nAverage Fertilizer Usage by Soil Type")

print(df.groupby("Soil_Type")["Fertilizer_kg"].mean())

print("\nEDA Completed Successfully.")

```

Output:

First Five Records

	Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
0	F01	850	28	60	Loamy	3200
1	F02	600	32	45	Sandy	2100
2	F03	900	27	65	Loamy	3500
3	F04	500	34	40	Sandy	1800
4	F05	780	29	55	Clay	3000

Last Five Records

	Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
15	F16	820	28	60	Clay	3150
16	F17	700	30	50	Clay	2700
17	F18	960	25	75	Loamy	3900
18	F19	540	34	41	Sandy	1850
19	F20	790	29	56	Clay	3050

Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 20 entries, 0 to 19

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Farm_ID	20 non-null	object
1	Rainfall_mm	20 non-null	int64
2	Temperature	20 non-null	int64
3	Fertilizer_kg	20 non-null	int64

```
4   Soil_Type      20 non-null    object
5   Crop_Yield_kg  20 non-null    int64
dtypes: int64(4), object(2)
memory usage: 1.1+ KB
None
```

```
Dataset Shape
(20, 6)
```

```
Column Names
Index(['Farm_ID', 'Rainfall_mm', 'Temperature', 'Fertilizer_kg',
      'Soil_Type',
      'Crop_Yield_kg'],
      dtype='object')
```

```
Missing Values
Farm_ID      0
Rainfall_mm  0
Temperature  0
Fertilizer_kg 0
Soil_Type    0
Crop_Yield_kg 0
dtype: int64
```

```
Duplicate Records
0
```

```
Statistical Summary
```

	Rainfall_mm	Temperature	Fertilizer_kg	Crop_Yield_kg
count	20.000000	20.000000	20.000000	20.000000
mean	743.000000	29.550000	54.750000	2820.000000
std	144.335136	2.762055	10.507516	681.407213
min	500.000000	25.000000	40.000000	1800.000000
25%	615.000000	27.750000	45.750000	2175.000000
50%	770.000000	29.000000	54.500000	2975.000000
75%	857.500000	32.000000	60.500000	3250.000000
max	960.000000	34.000000	75.000000	3900.000000

```
Mean
Rainfall_mm      743.00
Temperature       29.55
Fertilizer_kg     54.75
Crop_Yield_kg    2820.00
dtype: float64
```

```
Median
Rainfall_mm      770.0
Temperature       29.0
Fertilizer_kg     54.5
Crop_Yield_kg    2975.0
dtype: float64
```

```
Minimum
Rainfall_mm      500
Temperature       25
Fertilizer_kg     40
Crop_Yield_kg    1800
dtype: int64
```

```
Maximum
Rainfall_mm      960
```

Temperature 34
Fertilizer_kg 75
Crop_Yield_kg 3900
dtype: int64

Standard Deviation
Rainfall_mm 144.335136
Temperature 2.762055
Fertilizer_kg 10.507516
Crop_Yield_kg 681.407213
dtype: float64

Soil Type Count
Soil_Type
Loamy 7
Sandy 7
Clay 6
Name: count, dtype: int64

Average Crop Yield by Soil Type
Soil_Type
Clay 2941.666667
Loamy 3514.285714
Sandy 2021.428571
Name: Crop_Yield_kg, dtype: float64

Correlation Matrix

	Rainfall_mm	Temperature	Fertilizer_kg	Crop_Yield_kg
Rainfall_mm	1.000000	-0.993192	0.980550	0.995521
Temperature	-0.993192	1.000000	-0.979735	-0.993300
Fertilizer_kg	0.980550	-0.979735	1.000000	0.983181
Crop_Yield_kg	0.995521	-0.993300	0.983181	1.000000

Top 5 Highest Yield Farms

	Farm_ID	Soil_Type	Crop_Yield_kg
17	F18	Loamy	3900
13	F14	Loamy	3800
6	F07	Loamy	3700
2	F03	Loamy	3500
10	F11	Loamy	3400

=====

EDA SUMMARY

=====

Total Farms : 20

Highest Crop Yield
Farm_ID F18
Crop_Yield_kg 3900
Name: 17, dtype: object

Lowest Crop Yield
Farm_ID F04
Crop_Yield_kg 1800
Name: 3, dtype: object

Average Crop Yield by Soil Type
Soil_Type

```
Clay      2941.666667
Loamy     3514.285714
Sandy     2021.428571
Name: Crop_Yield_kg, dtype: float64
```

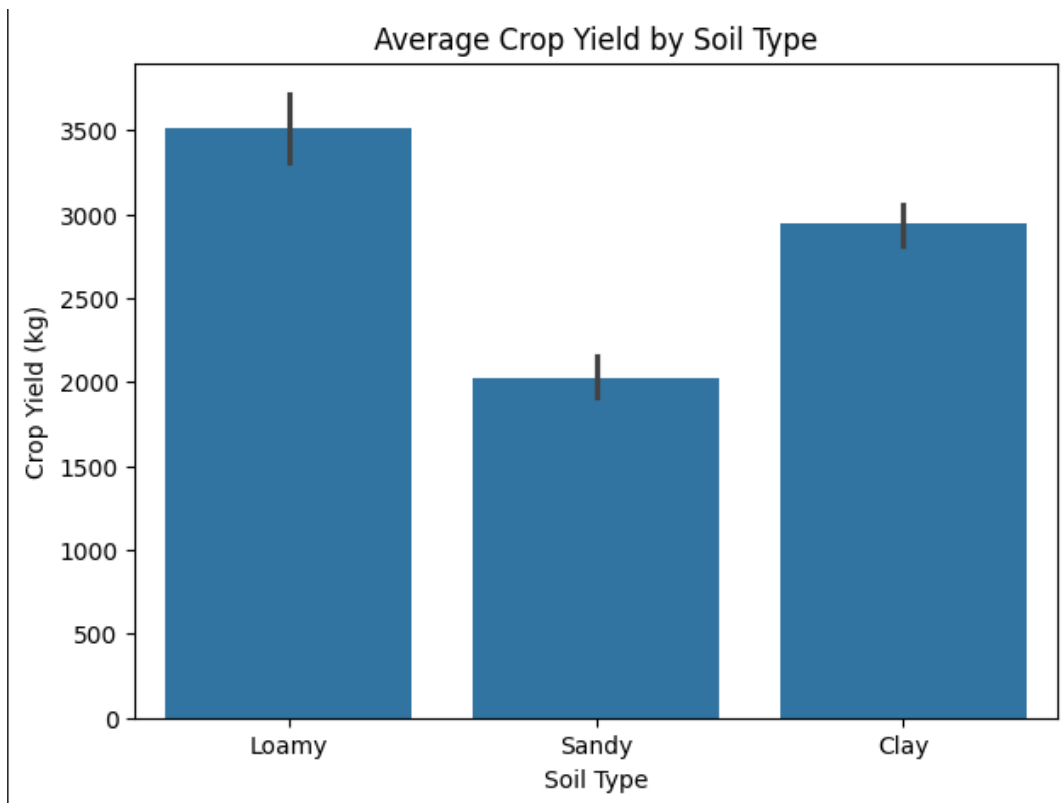
Average Rainfall by Soil Type

```
Soil_Type
Clay      761.666667
Loamy     892.857143
Sandy     577.142857
Name: Rainfall_mm, dtype: float64
```

Average Fertilizer Usage by Soil Type

```
Soil_Type
Clay      54.500000
Loamy     66.000000
Sandy     43.714286
Name: Fertilizer_kg, dtype: float64
```

EDA Completed Successfully.



Distribution of Numerical Features

